

39-payment-integration

Modul 39: Payment Integration (Midtrans, Xendit, Stripe)



Figure 1: Payment Integration

Level: Mahir

Estimasi Waktu: 12 jam

Prasyarat: Node.js + Express (Modul 6), pemahaman REST API & webhook

Output Akhir: Payment flow end-to-end: checkout → payment gateway → webhook → status update

□ Materi

Sesi	Topik
01	Payment Basics — flow overview, gateway concepts, sandbox vs production, Midtrans Snap API
02	Midtrans Core API & Xendit — charge, status, cancel, refund, webhook signature, error handling
03	Stripe Webhook — Checkout Session, Payment Intent, webhook security, idempotency
04	Production Payment — DB schema, idempotency & race condition, retry & reconciliation, PCI compliance

□ Output Akhir

Sistem payment end-to-end yang mencakup:

- **Flow checkout** — user pilih produk → hitung total → redirect ke payment gateway
- **Payment gateway integration** — Midtrans Snap, Midtrans Core API, Xendit Invoice, Stripe Checkout Session
- **Webhook handler** — verifikasi signature, update status transaksi, idempotency protection
- **Status update** — mapping status dari berbagai gateway ke status internal aplikasi
- **Reconciliation** — cron job sync status transaksi antara database dan payment gateway
- **Error handling & retry** — handle timeout, expired, denied, cancel scenarios

□ AI Prompt Exercises

Gunakan prompt ini di AI assistant seperti ChatGPT, Claude, atau Gemini untuk membantu pengembangan:

Prompt 1: Generate CRUD payment endpoint

Buat Express endpoint POST /api/payments/checkout

- Terima { productId, quantity, paymentMethod }
- Hitung total dari database produk
- Generate transactionId unik
- Return Snap transaction token (Midtrans)

- Simpan transaction_id, amount, status='pending' ke tabel transactions
- Pakai error handling async wrapper

Prompt 2: Generate webhook handler

Buat Express endpoint POST /api/payments/webhook/midtrans

- Verifikasi signature hash: sha512(order_id+status_code+gross_amount+server_key)
- Mapping status transaksi Midtrans ke status internal:
settlement->success, capture->success, pending->pending,
deny->failed, expire->expired, cancel->failed
- Update tabel transactions dengan status baru
- Return 200 OK
- Error handling: catch all, log, return 500

Prompt 3: Generate reconciliation cron

Buat cron job harian untuk reconciliation payment:

- Ambil semua transaksi status pending yg > 24 jam
- Cek status real ke Midtrans API (transaction.status())
- Update status di database jika berbeda
- Log hasil reconciliation (matched, updated, failed)
- Pakai node-cron

Prompt 4: Generate payment database schema

Buat migration SQL untuk tabel payments ecosystem:

- transactions: id, order_id, user_id, amount, status, payment_method, gateway, gateway_transaction_id, paid_at, created_at, updated_at
- refunds: id, transaction_id, amount, reason, status, gateway_refund_id, created_at
- invoices: id, transaction_id, invoice_number, due_date, paid_at, pdf_url, created_at
- Tambah index untuk order_id, gateway_transaction_id, status
- status enum: pending, success, failed, expired, refunded

Prompt 5: Generate idempotency middleware

Buat Express middleware idempotency:

- Baca header Idempotency-Key dari request
- Cek di Redis/sql: jika key sudah ada, return response cached
- Jika baru, proses request dan simpan response
- Expire cache dalam 24 jam
- Handle race condition dengan database lock
- Return 409 Conflict jika key masih diproses

Sesi 01: Payment Basics — Midtrans Snap Integration

Tujuan: Memahami payment flow, konsep payment gateway, perbedaan sandbox vs production, dan integrasi Midtrans Snap API.

□ Materi

1. Payment Flow Overview

Flow pembayaran online umum terdiri dari 4 tahap:

sequenceDiagram

participant User

participant App as Aplikasi (Backend)

participant PG as Payment Gateway

participant Bank

User->>App: 1. Pilih produk & checkout

App->>App: 2. Buat order (status: pending)

App->>PG: 3. Request payment (Snap Token)

PG-->>App: 4. Return Snap Token / Redirect URL

App-->>User: 5. Redirect ke halaman payment gateway

User->>PG: 6. Isi data pembayaran

PG->>Bank: 7. Proses pembayaran

Bank-->>PG: 8. Konfirmasi (success/fail)

PG-->>User: 9. Callback (halaman sukses/gagal)

PG->>App: 10. Webhook (notifikasi server-side)

App->>App: 11. Update status order

App-->>User: 12. Konfirmasi ke user (redirect / realtime)

Penjelasan:

- **Checkout (1-2):** User pilih produk, backend buat order dengan status pending
- **Payment Request (3-4):** Backend request token ke payment gateway
- **Redirect (5-6):** User diarahkan ke halaman payment gateway untuk bayar
- **Proses (7-8):** Gateway proses pembayaran via bank/VA/QRIS/dll
- **Callback (9):** Gateway redirect user kembali ke aplikasi (halaman sukses/gagal)
- **Webhook (10-11):** Gateway kirim notifikasi ke backend — **ini kritis** karena user bisa tutup browser sebelum callback

- **Konfirmasi (12):** Backend update status dan beri konfirmasi ke user

2. Payment Gateway Concepts

Konsep	Penjelasan
Payment Gateway	Layanan pihak ketiga yang memproses transaksi (Midtrans, Xendit, Stripe)
Snap	Midtrans UI payment — modal/redirect yang handle semua metode pembayaran
Core API	API langsung dari Midtrans untuk charge, status, cancel, refund
Direct Debit	Debit langsung dari rekening bank user
Virtual Account (VA)	Nomor virtual account untuk transfer bank
QRIS	Standar QR code pembayaran Indonesia
Invoice	URL pembayaran yang bisa dikirim ke user (Xendit)
Webhook	HTTP callback dari gateway ke server aplikasi untuk notifikasi real-time
Callback	Redirect dari gateway ke frontend setelah pembayaran selesai

3. Sandbox vs Production

Aspek	Sandbox	Production
Server Key	SB-Mid-server-xxx	Mid-server-xxx
Client Key	SB-Mid-client-xxx	Mid-client-xxx
Base URL	https://app.sandbox.midtrans.com	https://app.midtrans.com
Uang	Virtual — tidak real	Real
Kartu	Kartu test (4111 1111 1111 1111)	Kartu asli
Rate Limit	Lebih longgar	Strict
Webhook	Bisa test pake midtrans dashboard	Harus endpoint public (ngrok/deploy)

4. Midtrans Snap API — Setup & Redirect Flow

4.1 Instalasi

```
npm install midtrans-client
```

4.2 Konfigurasi Midtrans Client

```
// config/midtrans.js
const midtransClient = require('midtrans-client');

const snap = new midtransClient.Snap({
  isProduction: process.env.MIDTRANS_IS_PRODUCTION === 'true',
  serverKey: process.env.MIDTRANS_SERVER_KEY,
  clientKey: process.env.MIDTRANS_CLIENT_KEY,
});

const coreApi = new midtransClient.CoreApi({
  isProduction: process.env.MIDTRANS_IS_PRODUCTION === 'true',
  serverKey: process.env.MIDTRANS_SERVER_KEY,
  clientKey: process.env.MIDTRANS_CLIENT_KEY,
});

module.exports = { snap, coreApi };
```

4.3 Environment Variables

```
# .env
MIDTRANS_SERVER_KEY=SB-Mid-server-your-server-key
MIDTRANS_CLIENT_KEY=SB-Mid-client-your-client-key
MIDTRANS_IS_PRODUCTION=false
MIDTRANS_PRODUCTION_SERVER_KEY=Mid-server-your-production-key
MIDTRANS_PRODUCTION_CLIENT_KEY=Mid-client-your-production-key
```

4.4 Checkout Endpoint — Generate Snap Token

```
// routes/payment.js
const express = require('express');
const router = express.Router();
const { snap } = require('../config/midtrans');
const { v4: uuidv4 } = require('uuid');

// Simple in-memory db (ganti dengan PostgreSQL nanti)
const transactions = [];

router.post('/checkout', async (req, res) => {
  try {
    const { productId, quantity, paymentMethod } = req.body;

    // TODO: ambil harga dari database
    const productPrice = 150000;
    const grossAmount = productPrice * quantity;
```

```

const orderId = `ORDER-${Date.now()}-${uuidv4().slice(0, 8)}`;

const parameter = {
  transaction_details: {
    order_id: orderId,
    gross_amount: grossAmount,
  },
  credit_card: {
    secure: true,
  },
  customer_details: {
    first_name: 'Budi',
    last_name: 'Santoso',
    email: 'budi@example.com',
    phone: '081234567890',
  },
};

const transaction = await snap.createTransaction(parameter);

// Simpan transaksi ke database
transactions.push({
  order_id: orderId,
  transaction_id: transaction.token,
  gross_amount: grossAmount,
  status: 'pending',
  created_at: new Date(),
});

res.json({
  success: true,
  data: {
    token: transaction.token,
    redirect_url: transaction.redirect_url,
    order_id: orderId,
  },
});
} catch (error) {
  console.error('Midtrans checkout error:', error);
  res.status(500).json({
    success: false,
    message: 'Gagal memproses checkout',
    error: error.message,
  });
}
});

```

```
module.exports = router;
module.exports.transactions = transactions;
```

4.5 Frontend — Redirect to Snap

```
<!-- public/checkout.html -->
<!DOCTYPE html>
<html>
<head>
  <title>Checkout</title>
</head>
<body>
  <h1>Checkout</h1>
  <button id="pay-button">Bayar Sekarang</button>

  <script src="https://app.sandbox.midtrans.com/snap/snap.js"
    data-client-key="YOUR_CLIENT_KEY"></script>
  <script>
    document.getElementById('pay-button').onclick = function () {
      fetch('/api/payments/checkout', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          productId: 'PROD-001',
          quantity: 2,
          paymentMethod: 'credit_card',
        })
      })
        .then(res => res.json())
        .then(data => {
          if (data.success) {
            // Mode 1: Snap Modal (popup)
            snap.pay(data.data.token, {
              onSuccess: function (result) {
                console.log('Payment success:', result);
                window.location.href = '/success.html?order_id=' + data.data.order_id;
              },
              onPending: function (result) {
                console.log('Payment pending:', result);
              },
              onError: function (result) {
                console.error('Payment error:', result);
              },
              onClose: function () {
                console.log('User closed popup');
              }
            });
          }
        });
    };
  </script>
```



```

    },
  });
}
});
};
</script>
</body>
</html>

```

4.6 Callback & Redirect Page

```

// routes/payment.js – tambahkan
router.get('/status/:orderId', (req, res) => {
  const tx = transactions.find(t => t.order_id === req.params.orderId);
  if (!tx) {
    return res.status(404).json({ success: false, message: 'Transaksi tidak dite
  }
  res.json({ success: true, data: tx });
});

```

5. Webhook Endpoint (Sederhana)

```

// routes/webhook.js
const express = require('express');
const router = express.Router();

// Sementara – next session kita bahas verifikasi
router.post('/midtrans', (req, res) => {
  const notification = req.body;

  console.log('Webhook received:', notification);

  const { order_id, transaction_status, fraud_status } = notification;

  // Mapping status (detail di sesi 02)
  const statusMap = {
    capture: transaction_status === 'capture' && fraud_status === 'accept' ? 'su
    settlement: 'success',
    pending: 'pending',
    deny: 'failed',
    expire: 'expired',
    cancel: 'failed',
  };

  const newStatus = statusMap[transaction_status] || 'unknown';

```

```

// Update transaksi
const tx = transactions.find(t => t.order_id === order_id);
if (tx) {
  tx.status = newStatus;
  tx.updated_at = new Date();
  console.log(`Transaksi ${order_id} updated ke ${newStatus}`);
}

res.status(200).json({ status: 'ok' });
});

module.exports = router;

```

6. Integrasi ke app.js

```

// app.js
const express = require('express');
const paymentRoutes = require('./routes/payment');
const webhookRoutes = require('./routes/webhook');

const app = express();

// Webhook harus raw body – jangan pakai express.json() untuk /webhook
app.use('/api/webhook', express.raw({ type: 'application/json' })), webhookRoutes

// Routes lain pake express.json()
app.use(express.json());
app.use('/api/payments', paymentRoutes);
app.use(express.static('public'));

app.listen(3000, () => {
  console.log('Server running on port 3000');
});

```

Catatan: Webhook handler perlu `express.raw()` karena signature verification butuh body mentah (raw). Kalau pake `express.json()`, body sudah diubah dan signature verification gagal.

□ Latihan

Latihan 1: Setup Midtrans Sandbox

Buat akun Midtrans sandbox dan konfigurasi environment:

1. Daftar di [Midtrans Dashboard](#) (pilih sandbox)
2. Ambil Server Key dan Client Key dari menu Settings → Access Keys
3. Buat file .env dengan 4 variable Midtrans
4. Buat file config/midtrans.js export snap dan coreApi

Output: File .env terisi, file config/midtrans.js siap pakai.

Latihan 2: Checkout Endpoint

Buat POST /api/payments/checkout:

1. Terima { productId, quantity, paymentMethod }
2. Generate order_id format INV-{timestamp}-{random}
3. Set gross_amount dari harga produk (hardcode dulu)
4. Call snap.createTransaction() dengan parameter lengkap
5. Return { token, redirect_url, order_id }
6. Simpan transaksi ke array (in-memory)

Output: Endpoint berfungsi, test dengan curl/Postman dapat token.

Latihan 3: Frontend Snap Integration

Buat halaman HTML sederhana:

1. Tombol “Bayar Sekarang” trigger fetch ke /api/payments/checkout
2. Panggil snap.pay() dengan token dari response
3. Handle callback onSuccess → redirect ke /success.html?order_id=...
4. Handle onError dan onClose

Output: Flow lengkap: klik → fetch → popup snap → bayar → redirect.

Latihan 4: Webhook Endpoint Sederhana

Buat POST /api/webhook/midtrans:

1. Terima raw JSON body dari Midtrans
2. Log notification yang diterima
3. Map transaction_status ke status internal (pending/success/failed/expired)
4. Update status transaksi di array
5. Return 200 OK

Output: Webhook menerima notifikasi dan update status transaksi.

□ Ringkasan

- Payment flow: checkout → gateway redirect → user bayar → web-hook update status

- Snap API: paling mudah, handle UI pembayaran otomatis
- Sandbox vs Production: berbeda server key dan base URL
- Webhook: critical untuk update status real-time
- Jangan gunakan `express.json()` di route webhook — gunakan `express.raw()`

Next: Sesi 02 — Midtrans Core API & Xendit (charge, status, cancel, refund, webhook signature verification).

Sesi 02: Midtrans Core API & Xendit

Tujuan: Menguasai Midtrans Core API (charge, status, cancel, refund), Xendit API (invoice, VA, QRIS), transaction status mapping, webhook signature verification, dan error handling.

□ Materi

1. Midtrans Core API

Core API memberikan kontrol penuh dibanding Snap. Kita handle UI sendiri, dan panggil API langsung.

1.1 Charge — Proses Pembayaran

```
// services/midtrans-core.js
const { coreApi } = require('../config/midtrans');

/**
 * Charge dengan Virtual Account (BCA)
 */
async function chargeVA(orderId, amount, customer) {
  const parameter = {
    payment_type: 'bank_transfer',
    transaction_details: {
      order_id: orderId,
      gross_amount: amount,
    },
    bank_transfer: {
      bank: 'bca',
      va_number: '12345678901',
    },
    customer_details: {
      first_name: customer.firstName,
      last_name: customer.lastName,
    },
  };
}
```

```

        email: customer.email,
        phone: customer.phone,
    },
};

try {
    const chargeResponse = await coreApi.charge(parameter);
    return chargeResponse;
} catch (error) {
    console.error('Midtrans charge VA error:', error);
    throw error;
}
}

/**
 * Charge dengan QRIS
 */
async function chargeQRIS(orderId, amount, customer) {
    const parameter = {
        payment_type: 'gopay', // QRIS via GoPay
        transaction_details: {
            order_id: orderId,
            gross_amount: amount,
        },
        customer_details: {
            first_name: customer.firstName,
            last_name: customer.lastName,
            email: customer.email,
        },
    };

    const chargeResponse = await coreApi.charge(parameter);
    return chargeResponse;
}

```

```
module.exports = { chargeVA, chargeQRIS };
```

Response VA Charge:

```

{
  "status_code": "201",
  "status_message": "Success, Bank Transfer transaction is created",
  "transaction_id": "9e9ed7b0-5d7b-4c5d-9a7b-8f1e2d3c4b5a",
  "order_id": "ORDER-1712345678901-abc123",
  "gross_amount": "150000.00",
  "payment_type": "bank_transfer",
  "transaction_time": "2025-01-15 10:30:00",

```

```

    "transaction_status": "pending",
    "va_numbers": [
      {
        "bank": "bca",
        "va_number": "12345678901"
      }
    ]
  }
}

```

1.2 Transaction Status

```

async function getTransactionStatus(orderId) {
  try {
    const statusResponse = await coreApi.transaction.status(orderId);
    return statusResponse;
  } catch (error) {
    console.error('Status check error:', error);
    throw error;
  }
}

```

// Contoh penggunaan

```

// const status = await getTransactionStatus('ORDER-1712345678901-abc123');
// console.log(status.transaction_status); // 'settlement', 'pending', 'deny', a

```

1.3 Cancel Transaction

```

async function cancelTransaction(orderId) {
  try {
    const cancelResponse = await coreApi.transaction.cancel(orderId);
    return cancelResponse;
  } catch (error) {
    if (error.http_code === 412) {
      console.log('Transaksi sudah di-refund, tidak bisa cancel');
    }
    throw error;
  }
}

```

1.4 Refund Transaction

```

/**
 * Refund – hanya untuk transaksi yang sudah settlement
 */
async function refundTransaction(orderId, amount, reason) {
  const parameter = {

```

```

      refund_key: `refund-${orderId}-${Date.now()}`,
      amount: amount,
      reason: reason,
    };

    try {
      const refundResponse = await coreApi.transaction.refund(orderId, parameter);
      return refundResponse;
    } catch (error) {
      console.error('Refund error:', error);
      throw error;
    }
  }
}

```

2. Xendit API

Xendit cocok untuk pasar Indonesia dengan dukungan penuh VA, QRIS, dan Invoice.

2.1 Setup Xendit Client

```
npm install xendit-node
```

```

// config/xendit.js
const Xendit = require('xendit-node');

const xendit = new Xendit({
  secretKey: process.env.XENDIT_SECRET_KEY,
});

const { Invoice, VirtualAccount, QRCode } = xendit;

module.exports = { xendit, Invoice, VirtualAccount, QRCode };

# .env tambahan
XENDIT_SECRET_KEY=xnd_development_your-secret-key

```

2.2 Create Invoice (Xendit)

```

// services/xendit-invoice.js
const { Invoice } = require('../config/xendit');

async function createInvoice(orderId, amount, customer) {
  try {
    const invoice = await Invoice.create({
      external_id: orderId,

```

```

    amount: amount,
    payer_email: customer.email,
    description: `Pembayaran order ${orderId}`,
    customer: {
      given_names: customer.firstName,
      surname: customer.lastName,
      email: customer.email,
      mobile_number: customer.phone,
    },
    customer_notification_preference: {
      invoice_paid: ['email', 'whatsapp'],
      invoice_expired: ['email'],
    },
    success_redirect_url: 'https://yourapp.com/payment/success',
    failure_redirect_url: 'https://yourapp.com/payment/failed',
    currency: 'IDR',
  });

  return {
    id: invoice.id,
    invoice_url: invoice.invoice_url,
    external_id: invoice.external_id,
    amount: invoice.amount,
    status: invoice.status,
    expiry_date: invoice.expiry_date,
  };
} catch (error) {
  console.error('Xendit create invoice error:', error);
  throw error;
}
}

```

2.3 Create Virtual Account (Xendit)

```

async function createVA(externalId, amount, bank, customerName) {
  try {
    const va = await VirtualAccount.createFixedVA({
      external_id: externalId,
      bank_code: bank, // 'BCA', 'BNI', 'MANDIRI', 'BRI'
      name: customerName,
      is_single_use: true,
      expected_amount: amount,
      expiration_date: new Date(Date.now() + 24 * 60 * 60 * 1000), // 24 jam
    });

    return {

```



```

        id: va.id,
        bank: va.bank_code,
        va_number: va.account_number,
        external_id: va.external_id,
        amount: va.expected_amount,
        expiry: va.expiration_date,
    };
} catch (error) {
    console.error('Xendit create VA error:', error);
    throw error;
}
}

```

2.4 Create QRIS (Xendit)

```

async function createQRIS(externalId, amount) {
    try {
        const qrCode = await QRCode.create({
            external_id: externalId,
            type: 'DYNAMIC',
            amount: amount,
            callback_url: 'https://yourapp.com/api/webhook/xendit',
        });

        return {
            id: qrCode.id,
            qr_string: qrCode.qr_string,
            external_id: qrCode.external_id,
            amount: qrCode.amount,
        };
    } catch (error) {
        console.error('Xendit create QRIS error:', error);
        throw error;
    }
}

```

3. Transaction Status Mapping

Setiap payment gateway punya istilah status berbeda. Kita harus mapping ke status internal yang konsisten.

Status Internal	Midtrans	Xendit	Stripe
pending	pending	PENDING	requires_payment_method / re-quires_action
success	settlement, capture (fraud accept)	PAID, SETTLED	succeeded, complete
failed	deny, cancel	FAILED	requires_payment_method (failed attempt)
expired	expire	EXPIRED	—
refund	refund, partial_refund	—	refunded
void	—	VOIDED	canceled

Fungsi Mapping:

```
// utils/status-mapper.js
const PAYMENT_GATEWAYS = {
  MIDTRANS: 'midtrans',
  XENDIT: 'xendit',
  STRIPE: 'stripe',
};

const statusMap = {
  [PAYMENT_GATEWAYS.MIDTRANS]: {
    pending: 'pending',
    settlement: 'success',
    capture: (fraudStatus) =>
      fraudStatus === 'accept' ? 'success' : 'failed',
    deny: 'failed',
    expire: 'expired',
    cancel: 'failed',
    refund: 'refunded',
    partial_refund: 'partial_refund',
  },
  [PAYMENT_GATEWAYS.XENDIT]: {
    PENDING: 'pending',
    PAID: 'success',
    SETTLED: 'success',
    FAILED: 'failed',
    EXPIRED: 'expired',
    VOIDED: 'failed',
  },
};
```

```

function mapMidtransStatus(transactionStatus, fraudStatus) {
  const mapping = statusMap[PAYMENT_GATEWAYS.MIDTRANS][transactionStatus];
  if (typeof mapping === 'function') {
    return mapping(fraudStatus);
  }
  return mapping || 'unknown';
}

function mapXenditStatus(xenditStatus) {
  return statusMap[PAYMENT_GATEWAYS.XENDIT][xenditStatus] || 'unknown';
}

module.exports = { mapMidtransStatus, mapXenditStatus, PAYMENT_GATEWAYS };

```

4. Webhook Signature Verification

4.1 Midtrans Webhook Verification Midtrans mengirim signature dalam body notifikasi. Kita verifikasi dengan SHA512(order_id + status_code + gross_amount + server_key).

```

// middleware/verify-midtrans.js
const crypto = require('crypto');

function verifyMidtransWebhook(req, res, next) {
  // Webhook endpoint harus pakai express.raw() - body masih buffer
  const notification = JSON.parse(req.body.toString());
  const serverKey = process.env.MIDTRANS_SERVER_KEY;

  const {
    order_id,
    status_code,
    gross_amount,
    signature_key,
  } = notification;

  // Generate signature untuk verifikasi
  const hash = crypto
    .createHash('sha512')
    .update(order_id + status_code + gross_amount + serverKey)
    .digest('hex');

  if (hash !== signature_key) {
    console.error('Invalid signature - possible fraud:', {
      order_id,
      expected: hash,
    });
  }
}

```

```

        received: signature_key,
    });
    return res.status(403).json({
        message: 'Invalid signature',
    });
}

console.log('Signature verified for order:', order_id);
req.notification = notification;
next();
}

module.exports = verifyMidtransWebhook;

```

4.2 Xendit Webhook Verification Xendit menggunakan callback token yang diset di dashboard.

```

// middleware/verify-xendit.js
function verifyXenditWebhook(req, res, next) {
    const callbackToken = req.headers['x-callback-token'];
    const expectedToken = process.env.XENDIT_WEBHOOK_VERIFICATION_TOKEN;

    if (!callbackToken) {
        return res.status(403).json({ message: 'Missing callback token' });
    }

    if (callbackToken !== expectedToken) {
        console.error('Invalid Xendit callback token');
        return res.status(403).json({ message: 'Invalid callback token' });
    }

    req.notification = req.body;
    next();
}

module.exports = verifyXenditWebhook;

```

4.3 Webhook Handler Lengkap

```

// routes/webhook.js (lengkap)
const express = require('express');
const router = express.Router();
const verifyMidtrans = require('../middleware/verify-midtrans');
const { mapMidtransStatus } = require('../utils/status-mapper');

// Midtrans Webhook

```

```

router.post('/midtrans', verifyMidtrans, async (req, res) => {
  try {
    const notification = req.notification;
    const { order_id, transaction_status, fraud_status } = notification;

    const newStatus = mapMidtransStatus(transaction_status, fraud_status);

    // Update database
    // await db('transactions').where({ order_id }).update({ status: newStatus })

    console.log(`[Midtrans] Order ${order_id}: ${transaction_status} → ${newStatus}`);

    res.status(200).json({ status: 'ok' });
  } catch (error) {
    console.error('Webhook handler error:', error);
    // Tetap return 200 – jangan 500, karena Midtrans akan retry
    res.status(200).json({ status: 'ok', message: 'received but error' });
  }
});

// Xendit Webhook
router.post('/xendit', async (req, res) => {
  try {
    const notification = req.body;

    // Xendit kirim berbagai event
    // invoice: { status: 'PAID', external_id: '...', amount: ... }
    // va: { status: 'ACTIVE', ... }
    // qris: { status: 'COMPLETED', ... }

    if (notification.status === 'PAID' || notification.status === 'SETTLED') {
      // const status = mapXenditStatus(notification.status);
      // await db('transactions').where({ order_id: notification.external_id }).
    }

    res.status(200).json({ status: 'ok' });
  } catch (error) {
    console.error('Xendit webhook error:', error);
    res.status(200).json({ status: 'ok' });
  }
});

module.exports = router;

```

5. Error Handling

```
// utils/payment-error.js
class PaymentError extends Error {
  constructor(message, code, gateway, rawError) {
    super(message);
    this.name = 'PaymentError';
    this.code = code;           // 'TIMEOUT', 'INVALID_AMOUNT', 'DUPLICATE'
    this.gateway = gateway;     // 'midtrans', 'xendit', 'stripe'
    this.rawError = rawError;
  }
}

function handleMidtransError(error) {
  const { http_code, status_code, status_message } = error;

  if (http_code === 400) {
    return new PaymentError('Bad request – parameter tidak valid', 'INVALID_PARAMETER', 'midtrans');
  }
  if (http_code === 401) {
    return new PaymentError('Unauthorized – server key salah', 'UNAUTHORIZED', 'midtrans');
  }
  if (http_code === 404) {
    return new PaymentError('Transaksi tidak ditemukan', 'NOT_FOUND', 'midtrans');
  }
  if (http_code === 412) {
    return new PaymentError('Transaksi sudah di-refund/cancel', 'ALREADY_CANCELLED', 'midtrans');
  }
  if (http_code === 500 || http_code === 504) {
    return new PaymentError('Midtrans server error', 'GATEWAY_ERROR', 'midtrans');
  }

  return new PaymentError(
    status_message || 'Unknown error',
    `MIDTRANS_${status_code}`,
    'midtrans',
    error
  );
}

module.exports = { PaymentError, handleMidtransError };
```

Contoh penggunaan:

```
// routes/payment.js – cancel dengan error handling
router.post('/cancel/:orderId', async (req, res) => {
  try {
```

```

    const result = await cancelTransaction(req.params.orderId);
    res.json({ success: true, data: result });
  } catch (error) {
    const paymentError = handleMidtransError(error);

    if (paymentError.code === 'ALREADY_CANCELLED') {
      return res.status(409).json({ success: false, message: 'Transaksi sudah di' });
    }
    if (paymentError.code === 'GATEWAY_ERROR') {
      return res.status(502).json({ success: false, message: 'Midtrans sedang be' });
    }

    res.status(500).json({ success: false, message: paymentError.message });
  }
});

```

□ Latihan

Latihan 1: Charge VA Midtrans

Buat endpoint POST /api/payments/charge-va:

1. Terima { orderId, amount, bank, customer }
2. Panggil coreApi.charge() dengan payment_type bank_transfer
3. Pilih bank: bca, bni, mandiri, bri
4. Return VA number dan detail pembayaran
5. Simpan transaksi status 'pending'

Output: Endpoint return VA number, user bisa transfer ke VA tersebut.

Latihan 2: Xendit Invoice & VA

Buat 3 endpoint Xendit:

1. POST /api/payments/xendit/invoice — create Xendit invoice, return invoice_url
2. POST /api/payments/xendit/va — create fixed VA dengan expiry 24 jam
3. POST /api/payments/xendit/qris — create QRIS dinamis

Output: 3 endpoint siap, bisa generate invoice/VA/QRIS dari Xendit.

Latihan 3: Webhook Signature Verification

Implementasi verifikasi webhook:

1. Midtrans: verifikasi signature_key dengan SHA512

2. Xendit: verifikasi x-callback-token header
3. Return 403 jika signature tidak valid
4. Return 200 dengan { status: 'ok' } jika valid

Output: Webhook hanya diproses jika signature valid. Request palsu ditolak.

Latihan 4: Transaction Status Check & Mapping

Buat endpoint GET /api/payments/status/:orderId:

1. Cek status via Midtrans API (transaction.status())
2. Map status Midtrans ke status internal menggunakan fungsi mapper
3. Bandingkan dengan status di database
4. Jika berbeda, update database dengan status terbaru
5. Handle error: 404 jika transaksi tidak ditemukan di gateway

Output: Status transaksi bisa dicek real-time dari Midtrans, otomatis update DB jika ada perubahan.

□ Ringkasan

- Core API: kontrol penuh atas charge, status, cancel, refund
- Xendit: alternatif kuat untuk pasar Indonesia (Invoice, VA, QRIS)
- Status mapping: setiap gateway punya istilah berbeda → mapping ke status internal konsisten
- Signature verification: kritis untuk keamanan webhook
- Error handling: bedakan error berdasarkan http_code untuk response yang tepat

Next: Sesi 03 — Stripe Webhook (Checkout Session, Payment Intent, webhook security, idempotency).

Sesi 03: Stripe Webhook

Tujuan: Memahami integrasi Stripe (Checkout Session, Payment Intent), webhook Stripe dengan endpoint secret, event types, idempotency, dan keamanan webhook.

□ Materi

1. Stripe Setup

1.1 Instalasi & Konfigurasi

```
npm install stripe
```



```
// config/stripe.js
const Stripe = require('stripe');

const stripe = new Stripe(process.env.STRIPE_SECRET_KEY, {
  apiVersion: '2025-02-24.acacia', // gunakan versi terbaru stabil
});

module.exports = stripe;

# .env
STRIPE_SECRET_KEY=sk_test_...
STRIPE_PUBLISHABLE_KEY=pk_test_...
STRIPE_WEBHOOK_SECRET=whsec_...
```

1.2 Stripe API Version Selalu pakai versi API stabil terbaru. Cek di [Stripe API Changelog](#).

2. Checkout Session

Checkout Session adalah halaman pembayaran hosted oleh Stripe. User diarahkan ke URL Stripe, bayar, lalu redirect kembali.

```
// routes/stripe-checkout.js
const express = require('express');
const router = express.Router();
const stripe = require('../config/stripe');

router.post('/create-checkout-session', async (req, res) => {
  try {
    const { priceId, quantity = 1, customerEmail } = req.body;

    const session = await stripe.checkout.sessions.create({
      mode: 'payment',
      line_items: [
        {
          price: priceId, // price_xxx dari Stripe Dashboard
          quantity: quantity,
        },
      ],
      customer_email: customerEmail,
      success_url: `${process.env.APP_URL}/payment/success?session_id={CHECKOUT_SESSION_ID}`,
      cancel_url: `${process.env.APP_URL}/payment/cancel`,
      metadata: {
        order_id: `ORDER-${Date.now()}`,
        source: 'web',
      },
    });

    res.json(session);
  } catch (error) {
    res.status(500).json({ error });
  }
});
```

```

});

res.json({
  success: true,
  data: {
    session_id: session.id,
    url: session.url,
    metadata: session.metadata,
  },
});
} catch (error) {
  console.error('Stripe checkout error:', error);
  res.status(500).json({
    success: false,
    message: error.message,
  });
}
});

module.exports = router;

```

Response:

```

{
  "success": true,
  "data": {
    "session_id": "cs_test_...",
    "url": "https://checkout.stripe.com/c/pay/cs_test_...",
    "metadata": {
      "order_id": "ORDER-1712345678901",
      "source": "web"
    }
  }
}

```

Custom Domain (Optional) Stripe Checkout bisa pakai custom domain, tapi default domain Stripe sudah cukup untuk development.

3. Payment Intent

Payment Intent memberikan kontrol lebih (dibanding Checkout Session). Cocok untuk pembayaran dengan UI sendiri.

```

// routes/stripe-payment-intent.js
const express = require('express');
const router = express.Router();

```

```

const stripe = require('../config/stripe');

router.post('/create-payment-intent', async (req, res) => {
  try {
    const { amount, currency = 'idr', paymentMethodType = 'card' } = req.body;

    const paymentIntent = await stripe.paymentIntents.create({
      amount: Math.round(amount * 100), // amount dalam sen
      currency: currency.toLowerCase(), // 'idr', 'usd'
      payment_method_types: [paymentMethodType],
      metadata: {
        integration_check: 'accept_a_payment',
      },
    });

    res.json({
      success: true,
      data: {
        client_secret: paymentIntent.client_secret,
        payment_intent_id: paymentIntent.id,
        amount: paymentIntent.amount,
        currency: paymentIntent.currency,
        status: paymentIntent.status,
      },
    });
  } catch (error) {
    console.error('Create PaymentIntent error:', error);
    res.status(500).json({ success: false, message: error.message });
  }
});

```

Client-side dengan Stripe.js:

```

<script src="https://js.stripe.com/v3/"></script>
<script>
  const stripe = Stripe('pk_test_...');

  async function initialize() {
    const response = await fetch('/api/stripe/create-payment-intent', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ amount: 150000, currency: 'idr' }),
    });
    const { data } = await response.json();

    const elements = stripe.elements({ clientSecret: data.client_secret });
    const paymentElement = elements.create('payment');
  }

```

```

paymentElement.mount('#payment-element');

document.getElementById('submit').onclick = async () => {
  const { error } = await stripe.confirmPayment({
    elements,
    confirmParams: {
      return_url: 'https://yourapp.com/payment/success',
    },
  });

  if (error) {
    console.error('Payment failed:', error.message);
  }
};

initialize();
</script>

```

4. Stripe Webhook

4.1 Setup Webhook Endpoint

```

// routes/stripe-webhook.js
const express = require('express');
const router = express.Router();
const stripe = require('..../config/stripe');

// Stripe webhook – harus raw body, endpoint secret diverifikasi
router.post('/stripe', express.raw({ type: 'application/json' }), async (req, res) => {
  const sig = req.headers['stripe-signature'];
  const endpointSecret = process.env.STRIPE_WEBHOOK_SECRET;

  let event;

  // 1. Verifikasi signature
  try {
    event = stripe.webhooks.constructEvent(
      req.body, // raw body
      sig, // signature header
      endpointSecret
    );
  } catch (err) {
    console.error('Stripe webhook signature verification failed:', err.message);
    return res.status(400).send(`Webhook Error: ${err.message}`);
  }
}

```

```

// 2. Handle event berdasarkan tipe
try {
  switch (event.type) {
    case 'checkout.session.completed': {
      const session = event.data.object;
      await handleCheckoutCompleted(session);
      break;
    }
    case 'checkout.session.expired': {
      const session = event.data.object;
      await handleCheckoutExpired(session);
      break;
    }
    case 'payment_intent.succeeded': {
      const paymentIntent = event.data.object;
      await handlePaymentSucceeded(paymentIntent);
      break;
    }
    case 'payment_intent.payment_failed': {
      const paymentIntent = event.data.object;
      await handlePaymentFailed(paymentIntent);
      break;
    }
    default:
      console.log(`Unhandled event type: ${event.type}`);
  }

  res.json({ received: true });
} catch (error) {
  console.error(`Error processing event ${event.type}:`, error);
  res.status(200).json({ received: true, error: true });
}
});

async function handleCheckoutCompleted(session) {
  const { metadata, payment_intent, amount_total, currency } = session;

  const transaction = {
    order_id: metadata.order_id,
    stripe_session_id: session.id,
    stripe_payment_intent: payment_intent,
    amount: amount_total / 100, // dari sen ke rupiah
    currency: currency,
    status: 'success',
    paid_at: new Date(),
  };

```

```

};

// TODO: simpan ke database
// await db('transactions').where({ order_id: metadata.order_id }).update(transaction)
console.log('Checkout completed:', transaction);
}

async function handleCheckoutExpired(session) {
  // Update status ke expired
  console.log('Checkout expired:', session.id);
}

async function handlePaymentSucceeded(paymentIntent) {
  console.log('Payment succeeded:', paymentIntent.id);
}

async function handlePaymentFailed(paymentIntent) {
  console.log('Payment failed:', paymentIntent.id);
}

module.exports = router;

```

4.2 Event Types Penting

Event	Trigger	Aksi
checkout.session.completed	Update status selesai bayar via Checkout Session	Update status success, kirim konfirmasi
checkout.session.expired	Sesi tidak dibayar dalam 24 jam	Update status expired
checkout.session.async_payment_succeeded	Pembayaran berhasil sukses (e.g., transfer bank)	Update status success
checkout.session.async_payment_failed	Pembayaran gagal	Update status failed
payment_intent.succeeded	Payment Intent sukses	Update status success
payment_intent.payment_failed	Payment Intent gagal	Update status failed
payment_intent.amount_capturable_updated	Pembayaran bisa diupdate capture	Update status pending

5. Webhook Security

5.1 Signature Verification Stripe menggunakan HMAC-SHA256 untuk signature. `stripe.webhooks.constructEvent()` handle verifikasi:

- Baca header `stripe-signature`
- Parse timestamp dan signature
- Generate HMAC dengan endpoint secret
- Compare signature
- **Replay protection:** cek timestamp dalam tolerance (default 5 menit)

// Manual signature verification (jika perlu)

```
const crypto = require('crypto');
```

```
function verifyStripeSignature(payload, sigHeader, secret) {  
  const parts = sigHeader.split(',');  
  const timestamp = parts.find(p => p.startsWith('t=')).split('=')[1];  
  const signatures = parts.filter(p => p.startsWith('v1=')).map(p => p.split('=')  
    .pop());  
  
  const signedPayload = `${timestamp}.${payload.toString()}`;  
  const expectedSig = crypto  
    .createHmac('sha256', secret)  
    .update(signedPayload)  
    .digest('hex');  
  
  if (!signatures.includes(expectedSig)) {  
    throw new Error('Invalid signature');  
  }  
  
  // Replay protection – cek timestamp dalam 5 menit  
  const now = Math.floor(Date.now() / 1000);  
  if (Math.abs(now - parseInt(timestamp)) > 300) {  
    throw new Error('Timestamp out of tolerance – possible replay attack');  
  }  
  
  return true;  
}
```

5.2 Replay Protection Apa itu replay attack?

sequenceDiagram

participant Attacker

participant App as Aplikasi (Webhook)

participant Stripe

Stripe->>App: Webhook: payment_intent.succeeded (legitimate)
App->>App: Process: update status success ☐

Attacker->>App: Webhook DUPLICATE: same payload + signature
Note over App: Tanpa replay protection → status success lagi ☐
App->>App: Process lagi: duplicate update

Note over App: Dengan timestamp check → $t=1720000000$ vs $now=1720001000$

Cara Stripe handle: 1. t = timestamp: waktu Stripe generate event
2. Webhook endpoint cek: $now - t \leq 300$ detik 3. Jika lebih dari 5 menit → tolak

6. Idempotency Key

Idempotency mencegah duplicate processing dari event yang sama.

6.1 Stripe Native Idempotency Stripe API mendukung idempotency key via header:

```
// Stripe otomatis handle idempotency untuk API calls
// Tapi untuk webhook handler, kita harus handle sendiri
async function createPaymentIntentWithIdempotency(amount, idempotencyKey) {
  try {
    const paymentIntent = await stripe.paymentIntents.create(
      {
        amount: amount,
        currency: 'idr',
      },
      {
        idempotencyKey: idempotencyKey,
      }
    );
    return paymentIntent;
  } catch (error) {
    if (error.type === 'StripeIdempotencyError') {
      console.log('Idempotency key conflict:', error.message);
      // Key sudah digunakan dengan parameter berbeda
    }
    throw error;
  }
}
```

6.2 Webhook Idempotency — Manual Untuk webhook, kita harus simpan event.id yang sudah diproses:


```

// middleware/idempotency.js
/**
 * Middleware idempotency untuk webhook
 * Menyimpan event_id yang sudah diproses di Redis/DB
 */
async function webhookIdempotency(req, res, next) {
  // Assumes event.id tersedia setelah signature verification
  if (req.body && req.body.id) {
    const eventId = req.body.id;

    // Cek apakah event sudah diproses
    const processed = await checkEventProcessed(eventId);
    if (processed) {
      console.log(`Duplicate webhook event: ${eventId} - skipping`);
      // Return 200 biaya Stripe tidak retry
      return res.status(200).json({ received: true, duplicate: true });
    }

    // Tandai sedang diproses (lock)
    await markEventProcessing(eventId);
    next();

    // Setelah response, tandai selesai
    res.on('finish', async () => {
      if (res.statusCode === 200) {
        await markEventProcessed(eventId);
      } else {
        await clearEventLock(eventId); // unlock untuk retry
      }
    });
  } else {
    next();
  }
}

// — Implementasi dengan Redis —
const redis = require('redis');
const client = redis.createClient();

async function checkEventProcessed(eventId) {
  const result = await client.get(`webhook:${eventId}`);
  return result === 'done';
}

async function markEventProcessing(eventId) {
  // SET NX – Redis atomic: set jika belum ada

```

```

    const set = await client.set(`webhook:${eventId}`, 'processing', {
      NX: true,
      EX: 60, // expire 60 detik – safety jika handler crash
    });
    return set === 'OK';
  }

  async function markEventProcessed(eventId) {
    await client.set(`webhook:${eventId}`, 'done', {
      EX: 86400, // expire 24 jam
    });
  }

  async function clearEventLock(eventId) {
    await client.del(`webhook:${eventId}`);
  }

```

6.3 Idempotency di Database (SQL) Alternatif tanpa Redis — pakai unique constraint:

```

-- Buat tabel untuk menyimpan processed events
CREATE TABLE processed_webhooks (
  id SERIAL PRIMARY KEY,
  event_id VARCHAR(255) NOT NULL UNIQUE,
  gateway VARCHAR(50) NOT NULL,
  event_type VARCHAR(100),
  status VARCHAR(20) DEFAULT 'processing', -- processing | done | failed
  processed_at TIMESTAMP DEFAULT NOW(),
  UNIQUE(event_id)
);

-- Index untuk lookup cepat
CREATE INDEX idx_processed_webhooks_event_id ON processed_webhooks(event_id);

// Implementasi dengan PostgreSQL
async function processEventUnique(eventId, gateway, processingFn) {
  const client = await pool.connect();

  try {
    await client.query('BEGIN');

    // Insert dengan ON CONFLICT – atomic
    const result = await client.query(
      `INSERT INTO processed_webhooks (event_id, gateway, status)
      VALUES ($1, $2, 'processing')
      ON CONFLICT (event_id) DO UPDATE SET status = processed_webhooks.status`
    );
  }

```

```

        RETURNING status`,
        [eventId, gateway]
    );

    // Jika sudah 'done', skip
    if (result.rows[0].status === 'done') {
        await client.query('ROLLBACK');
        return { skipped: true };
    }

    // Proses event
    const resultData = await processingFn();

    // Tandai selesai
    await client.query(
        `UPDATE processed_webhooks SET status = 'done' WHERE event_id = $1`,
        [eventId]
    );

    await client.query('COMMIT');
    return resultData;
} catch (error) {
    await client.query('ROLLBACK');
    throw error;
} finally {
    client.release();
}
}

```

□ Latihan

Latihan 1: Stripe Checkout Session

Buat endpoint POST `/api/stripe/create-checkout-session`:

1. Setup Stripe dengan secret key (mode test)
2. Buat Checkout Session dengan 1 produk (hardcode priceId)
3. Set success_url dan cancel_url
4. Tambahkan metadata order_id
5. Return session URL

Output: Endpoint return URL Stripe Checkout. Buka di browser, bisa bayar dengan kartu test Stripe (4242 4242 4242 4242).

Latihan 2: Stripe Payment Intent + Client

Buat flow pembayaran dengan Payment Intent:

1. Backend: `POST /api/stripe/create-payment-intent` — `return client_secret`
2. Frontend: render Stripe Elements (Card / Payment Element)
3. Client-side: `confirmPayment` dengan Stripe.js
4. Handle success dan error di frontend

Output: Form pembayaran dengan Stripe Elements, bisa input kartu dan bayar.

Latihan 3: Stripe Webhook Handler

Buat webhook endpoint `POST /api/webhook/stripe`:

1. Pakai `express.raw()` — jangan `express.json()`
2. Verifikasi signature dengan `stripe.webhooks.constructEvent()`
3. Handle 3 event types: `checkout.session.completed`, `payment_intent.succeeded`, `payment_intent.payment_failed`
4. Untuk setiap event, log detail dan return 200

Output: Webhook menerima event dari Stripe, memverifikasi signature, dan memproses event.

Latihan 4: Idempotency with Redis/DB

Implementasi idempotency di webhook handler:

1. Buat middleware `webhookIdempotency` yang cek `event_id`
2. Cek apakah `event_id` sudah diproses (di Redis atau DB)
3. Jika sudah → return 200 (skip)
4. Jika baru → proses, simpan `event_id`
5. Test dengan mengirim event Stripe yang sama 2 kali

Output: Event Stripe yang sama hanya diproses sekali. Request duplikat di-skip.

□ Ringkasan

- Checkout Session: hosted payment page — paling mudah diintegrasikan
- Payment Intent: kontrol lebih — cocok untuk custom UI
- Webhook security: `constructEvent` verifikasi signature + replay protection
- Event types: `checkout.session.completed`, `payment_intent.succeeded`, dll
- Idempotency: critical di production untuk mencegah duplicate processing
- Redis/DB idempotency: simpan `event_id` yang sudah diproses

Next: Sesi 04 — Production Payment (DB schema, idempotency & race condition, retry & reconciliation, PCI compliance).

Sesi 04: Production Payment

Tujuan: Merancang database payment yang robust, handle idempotency & race condition, implementasi retry & reconciliation, webhook timeout & retry pattern, dan memahami PCI compliance.

□ Materi

1. Payment Database Schema

Schema database untuk sistem payment yang production-ready.

1.1 Migration SQL

```
-- migrations/001_create_payment_tables.sql

-- Enum untuk status transaksi
CREATE TYPE transaction_status AS ENUM (
    'pending',
    'success',
    'failed',
    'expired',
    'refunded',
    'partial_refund'
);

-- Enum untuk payment method
CREATE TYPE payment_method AS ENUM (
    'credit_card',
    'bank_transfer',
    'virtual_account',
    'qris',
    'gopay',
    'shopeepay',
    'invoice'
);

-- Enum untuk gateway
CREATE TYPE payment_gateway AS ENUM (
    'midtrans',
```

```

        'xendit',
        'stripe'
    );

-- Tabel utama: transactions
CREATE TABLE transactions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    order_id VARCHAR(100) NOT NULL UNIQUE,
    user_id UUID NOT NULL REFERENCES users(id),
    amount BIGINT NOT NULL, -- dalam sen (IDR 15000 = 1500000)
    currency VARCHAR(3) NOT NULL DEFAULT 'IDR',
    status transaction_status NOT NULL DEFAULT 'pending',
    payment_method payment_method,
    gateway payment_gateway NOT NULL,
    gateway_transaction_id VARCHAR(255),
    gateway_response JSONB, -- raw response dari gateway
    paid_at TIMESTAMP WITH TIME ZONE,
    expired_at TIMESTAMP WITH TIME ZONE,
    created_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW()
);

-- Tabel: refunds
CREATE TABLE refunds (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    transaction_id UUID NOT NULL REFERENCES transactions(id),
    amount BIGINT NOT NULL,
    reason TEXT NOT NULL,
    status VARCHAR(20) NOT NULL DEFAULT 'pending', -- pending, success, failed
    gateway_refund_id VARCHAR(255),
    refund_key VARCHAR(255) UNIQUE, -- idempotency key untuk refund
    created_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW()
);

-- Tabel: invoices (untuk Xendit Invoice / manual invoice)
CREATE TABLE invoices (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    transaction_id UUID NOT NULL REFERENCES transactions(id),
    invoice_number VARCHAR(50) NOT NULL UNIQUE,
    due_date DATE NOT NULL,
    paid_at TIMESTAMP WITH TIME ZONE,
    pdf_url TEXT,
    notes TEXT,
    created_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW()
);

```

```

-- Tabel: webhook_events (idempotency + audit trail)
CREATE TABLE webhook_events (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  event_id VARCHAR(255) NOT NULL UNIQUE,
  gateway payment_gateway NOT NULL,
  event_type VARCHAR(100) NOT NULL,
  status VARCHAR(20) NOT NULL DEFAULT 'received', -- received, processing, processed
  raw_body JSONB,
  processed_at TIMESTAMP WITH TIME ZONE,
  error_message TEXT,
  created_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW()
);

-- Tabel: payment_attempts (retry log)
CREATE TABLE payment_attempts (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  transaction_id UUID NOT NULL REFERENCES transactions(id),
  attempt_number INT NOT NULL DEFAULT 1,
  gateway payment_gateway NOT NULL,
  request_body JSONB,
  response_body JSONB,
  status_code INT,
  success BOOLEAN NOT NULL DEFAULT false,
  error_message TEXT,
  attempted_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW()
);

-- === INDEXES ===
CREATE INDEX idx_transactions_order_id ON transactions(order_id);
CREATE INDEX idx_transactions_user_id ON transactions(user_id);
CREATE INDEX idx_transactions_status ON transactions(status);
CREATE INDEX idx_transactions_gateway_tx_id ON transactions(gateway_transaction_id);
CREATE INDEX idx_transactions_paid_at ON transactions(paid_at);

CREATE INDEX idx_refunds_transaction_id ON refunds(transaction_id);
CREATE INDEX idx_refunds_refund_key ON refunds(refund_key);

CREATE INDEX idx_webhook_events_event_id ON webhook_events(event_id);
CREATE INDEX idx_webhook_events_gateway ON webhook_events(gateway);
CREATE INDEX idx_webhook_events_status ON webhook_events(status);

CREATE INDEX idx_payment_attempts_transaction_id ON payment_attempts(transaction_id);

-- Trigger untuk auto-update updated_at
CREATE OR REPLACE FUNCTION update_updated_at_column()

```

```

RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = NOW();
    RETURN NEW;
END;
$$ language 'plpgsql';

CREATE TRIGGER update_transactions_updated_at
BEFORE UPDATE ON transactions
FOR EACH ROW
EXECUTE FUNCTION update_updated_at_column();

CREATE TRIGGER update_refunds_updated_at
BEFORE UPDATE ON refunds
FOR EACH ROW
EXECUTE FUNCTION update_updated_at_column();

```

1.2 Entity Relationship Diagram

```

erDiagram
    users ||--o{ transactions : has
    transactions ||--o{ refunds : has
    transactions ||--o{ invoices : has
    transactions ||--o{ payment_attempts : has

    users {
        uuid id PK
        string name
        string email
    }

    transactions {
        uuid id PK
        string order_id UK
        uuid user_id FK
        bigint amount
        string currency
        enum status
        enum payment_method
        enum gateway
        string gateway_transaction_id
        jsonb gateway_response
        timestamp paid_at
        timestamp expired_at
        timestamp created_at
    }

```



```

refunds {
  uuid id PK
  uuid transaction_id FK
  bigint amount
  string reason
  string status
  string gateway_refund_id
  string refund_key UK
}

invoices {
  uuid id PK
  uuid transaction_id FK
  string invoice_number UK
  date due_date
  timestamp paid_at
  string pdf_url
}

webhook_events {
  uuid id PK
  string event_id UK
  enum gateway
  string event_type
  string status
  jsonb raw_body
  timestamp processed_at
}

payment_attempts {
  uuid id PK
  uuid transaction_id FK
  int attempt_number
  enum gateway
  jsonb request_body
  jsonb response_body
  int status_code
  bool success
  timestamp attempted_at
}

```

2. Idempotency & Race Condition

2.1 Race Condition di Webhook Webhook bisa tiba bersamaan dari gateway, atau Stripe retry mengirim event yang sama.

sequenceDiagram

```
participant PG as Payment Gateway
participant S1 as Server Instance 1
participant S2 as Server Instance 2
participant DB as Database
```

PG->>S1: Webhook: payment_intent.succeeded (event_1)

PG->>S2: Webhook: payment_intent.succeeded (event_1) – duplicate

S1->>DB: SELECT status FROM transactions WHERE order_id='X'

S2->>DB: SELECT status FROM transactions WHERE order_id='X'

Note over S1,S2: Kedua instance baca status='pending'!

S1->>DB: UPDATE status='success'

S2->>DB: UPDATE status='success'

Note over S1,S2: □ Double update – masih aman (success)

PG->>S1: Webhook: payment_intent.refunded (event_2)

PG->>S2: Webhook: payment_intent.payment_failed (event_3) – tiba duluan

S1->>DB: SELECT status → 'success'

S2->>DB: SELECT status → 'success'

Note over S1,S2: Kedua instance validasi lolos!

S1->>DB: UPDATE status='refunded'

S2->>DB: UPDATE status='failed'

Note over S1,S2: □ Race! Status akhir 'failed' padahal seharusnya 'refunded'

2.2 Solusi: Optimistic Lock + Atomic Update

```
// services/payment-update.js
```

```
const pool = require('../config/database');
```

```
class PaymentUpdateService {
```

```
  /**
```

```
   * Update status transaksi dengan optimistic lock
```

```
   * Menggunakan WHERE clause dengan current_status untuk mencegah race
```

```

*/
async updateTransactionStatus(orderId, newStatus, expectedStatus) {
  const client = await pool.connect();

  try {
    await client.query('BEGIN');

    // Atomic update – hanya update jika status masih expected
    const result = await client.query(
      `UPDATE transactions
      SET status = $1, updated_at = NOW()
      WHERE order_id = $2 AND status = $3
      RETURNING id, status, order_id`,
      [newStatus, orderId, expectedStatus]
    );

    if (result.rows.length === 0) {
      // Status sudah berubah – kemungkinan race condition
      const current = await client.query(
        `SELECT status FROM transactions WHERE order_id = $1`,
        [orderId]
      );

      await client.query('COMMIT');

      return {
        updated: false,
        currentStatus: current.rows[0]?.status,
        message: `Status conflict: expected ${expectedStatus}, current ${current.rows[0]?.status}`
      };
    }

    await client.query('COMMIT');

    return {
      updated: true,
      currentStatus: newStatus,
    };
  } catch (error) {
    await client.query('ROLLBACK');
    throw error;
  } finally {
    client.release();
  }
}

```

```

/**
 * Webhook handler dengan idempotency + race condition protection
 */
async handleWebhookEvent(eventId, gateway, processFn) {
  const client = await pool.connect();

  try {
    await client.query('BEGIN');

    // Atomic insert – skip jika event sudah diproses
    const insertResult = await client.query(
      `INSERT INTO webhook_events (event_id, gateway, status)
      VALUES ($1, $2, 'processing')
      ON CONFLICT (event_id) DO UPDATE
      SET status = webhook_events.status
      WHERE webhook_events.status = 'processed'
      RETURNING status`,
      [eventId, gateway]
    );

    const currentStatus = insertResult.rows[0].status;

    // Jika sudah processed, skip
    if (currentStatus === 'processed') {
      await client.query('COMMIT');
      return { duplicate: true };
    }

    // Eksekusi handler dengan transaksi yang sama
    await processFn(client);

    // Tandai selesai
    await client.query(
      `UPDATE webhook_events SET status = 'processed', processed_at = NOW()
      WHERE event_id = $1`,
      [eventId]
    );

    await client.query('COMMIT');
    return { duplicate: false };
  } catch (error) {
    await client.query('ROLLBACK');
    throw error;
  } finally {
    client.release();
  }
}

```

```

    }
  }

  module.exports = new PaymentUpdateService();

```

2.3 Penggunaan

// routes/webhook.js – Stripe webhook dengan race protection

```

router.post('/stripe', verifyStripe, async (req, res) => {
  const event = req.stripeEvent;

  await paymentUpdate.handleWebhookEvent(
    event.id,
    'stripe',
    async (client) => {
      // Di sini kita sudah dalam transaksi yang sama
      // Semua query aman dari race condition

      if (event.type === 'checkout.session.completed') {
        const session = event.data.object;
        const orderId = session.metadata.order_id;

        // Atomic update dengan optimistic lock
        const result = await client.query(
          `UPDATE transactions
           SET status = 'success',
             gateway_transaction_id = $2,
             paid_at = NOW(),
             updated_at = NOW()
           WHERE order_id = $1 AND status = 'pending'
           RETURNING id`,
          [orderId, session.payment_intent]
        );

        if (result.rows.length === 0) {
          throw new Error(`Order ${orderId} sudah diupdate atau tidak ditemukan`);
        }
      }
    }
  );

  res.json({ received: true });
});

```

3. Retry & Reconciliation

3.1 Payment Attempt Retry Pattern

```
// services/retry-payment.js
const pool = require('../config/database');
const { stripe } = require('../config/stripe');
const { coreApi } = require('../config/midtrans');

const MAX_RETRIES = 3;
const RETRY_DELAYS = [60, 300, 900]; // 1 menit, 5 menit, 15 menit (detik)

async function retryFailedPayment(transactionId) {
  const client = await pool.connect();

  try {
    // Ambil transaksi
    const txResult = await client.query(
      `SELECT * FROM transactions WHERE id = $1`,
      [transactionId]
    );
    const tx = txResult.rows[0];

    if (!tx) throw new Error('Transaksi tidak ditemukan');
    if (tx.status !== 'failed') throw new Error(`Status ${tx.status} – bukan failed`);

    // Cek jumlah attempt sebelumnya
    const attemptCount = await client.query(
      `SELECT COUNT(*) as count FROM payment_attempts
      WHERE transaction_id = $1 AND success = false`,
      [transactionId]
    );

    const attemptNumber = parseInt(attemptCount.rows[0].count) + 1;

    if (attemptNumber > MAX_RETRIES) {
      throw new Error(`Max retries reached (${MAX_RETRIES}) untuk ${transactionId}`);
    }

    // Delay sesuai attempt number
    const delay = RETRY_DELAYS[attemptNumber - 1] || RETRY_DELAYS[RETRY_DELAYS.length - 1];
    await new Promise(resolve => setTimeout(resolve, delay * 1000));

    // Coba charge ulang
    let chargeResult;
    if (tx.gateway === 'stripe') {
```

```

        chargeResult = await stripe.paymentIntents.create({
            amount: tx.amount,
            currency: tx.currency.toLowerCase(),
        });
    } else if (tx.gateway === 'midtrans') {
        chargeResult = await coreApi.charge({
            payment_type: 'bank_transfer',
            transaction_details: {
                order_id: tx.order_id,
                gross_amount: tx.amount / 100,
            },
        });
    }
}

// Simpan attempt log
await client.query(
    `INSERT INTO payment_attempts (transaction_id, attempt_number, gateway,
    response_body, status_code, success, attempted_at)
    VALUES ($1, $2, $3, $4, $5, $6, NOW())`,
    [transactionId, attemptNumber, tx.gateway,
    JSON.stringify(chargeResult), 200, true]
);

// Update status
await client.query(
    `UPDATE transactions SET status = 'pending', updated_at = NOW()
    WHERE id = $1`,
    [transactionId]
);

return { retried: true, attemptNumber };
} catch (error) {
    // Log failed attempt
    await client.query(
        `INSERT INTO payment_attempts (transaction_id, attempt_number, gateway,
        error_message, success, attempted_at)
        VALUES ($1, $2, $3, $4, false, NOW())`,
        [transactionId, (attemptNumber || 1), tx?.gateway, error.message]
    );
    throw error;
} finally {
    client.release();
}
}

```

3.2 Reconciliation Cron Job

```
// cron/reconciliation.js
const cron = require('node-cron');
const pool = require('../config/database');
const { coreApi } = require('../config/midtrans');
const { default: Stripe } = require('stripe');
const stripe = require('../config/stripe');

// Jalankan setiap jam
cron.schedule('0 * * * *', async () => {
  console.log('[Reconciliation] Starting...');
  const stats = { matched: 0, updated: 0, failed: 0, errors: [] };

  try {
    // Ambil transaksi pending > 1 jam dan hari ini
    const pendingTx = await pool.query(
      `SELECT * FROM transactions
      WHERE status = 'pending'
      AND created_at > NOW() - INTERVAL '24 hours'
      AND created_at < NOW() - INTERVAL '1 hour'
      ORDER BY created_at ASC
      LIMIT 50`
    );

    for (const tx of pendingTx.rows) {
      try {
        let gatewayStatus;

        // Cek status ke gateway
        if (tx.gateway === 'midtrans') {
          const response = await coreApi.transaction.status(tx.order_id);
          gatewayStatus = response.transaction_status;
        } else if (tx.gateway === 'stripe') {
          if (tx.gateway_transaction_id) {
            const pi = await stripe.paymentIntents.retrieve(tx.gateway_transaction_id);
            gatewayStatus = pi.status;
          }
        }

        // Map ke status internal
        const internalStatus = mapGatewayStatusToInternal(tx.gateway, gatewayStatus);

        if (internalStatus && internalStatus !== 'pending') {
          // Update jika berbeda
          await pool.query(

```



```

        `UPDATE transactions
        SET status = $1, updated_at = NOW(),
            paid_at = CASE WHEN $1 = 'success' THEN NOW() ELSE paid_at END
        WHERE id = $2 AND status = 'pending`,
        [internalStatus, tx.id]
    );

    stats.updated++;
    console.log(`[Reconciliation] ${tx.order_id}: pending → ${internalStat
} else {
    stats.matched++;
}
} catch (txError) {
    stats.failed++;
    stats.errors.push({ order_id: tx.order_id, error: txError.message });
    console.error(`[Reconciliation] Error ${tx.order_id}:`, txError.message)
}
}

    console.log('[Reconciliation] Complete:', stats);
} catch (error) {
    console.error('[Reconciliation] Fatal error:', error);
}
});

function mapGatewayStatusToInternal(gateway, status) {
    const mapping = {
        midtrans: {
            settlement: 'success',
            capture: 'success',
            deny: 'failed',
            expire: 'expired',
            cancel: 'failed',
        },
        stripe: {
            succeeded: 'success',
            requires_payment_method: 'failed',
            canceled: 'failed',
        },
    };
    return mapping[gateway]?.[status] || null;
}

```

4. Webhook Timeout & Retry Pattern

Payment gateway biasanya punya timeout pendek untuk webhook (5-10 detik). Jika handler lama, gateway akan retry.

```
// middleware/webhook-queue.js
const EventEmitter = require('events');

/**
 * Webhook queue – proses webhook secara async
 * Response langsung 200, proses di background
 *
 * Cocok untuk:
 * - Handler yang perlu call external API lambat
 * - Kirim email/SMS notifikasi
 * - Generate invoice PDF
 */
class WebhookQueue extends EventEmitter {
  constructor() {
    super();
    this.queue = [];
    this.processing = false;
    this.maxConcurrent = 3;
    this.active = 0;
  }

  /**
   * Enqueue webhook untuk diproses async
   * Return 200 langsung agar gateway tidak retry
   */
  async enqueue(eventId, gateway, processFn) {
    const promise = new Promise((resolve, reject) => {
      this.queue.push({ eventId, gateway, processFn, resolve, reject });
    });

    this.processQueue();
    return promise;
  }

  async processQueue() {
    if (this.processing || this.active >= this.maxConcurrent) return;
    this.processing = true;

    while (this.queue.length > 0 && this.active < this.maxConcurrent) {
      const item = this.queue.shift();
      this.active++;
    }
  }
}
```

```

    // Proses tanpa await – biar concurrent
    item.processFn()
      .then((result) => {
        item.resolve(result);
        console.log(`[WebhookQueue] ${item.eventId} processed`);
      })
      .catch((error) => {
        item.reject(error);
        console.error(`[WebhookQueue] ${item.eventId} failed:`, error.message);
      })
      .finally(() => {
        this.active--;
        this.processQueue();
      });
  }

  this.processing = false;
}

const webhookQueue = new WebhookQueue();

// Penggunaan di route
router.post('/stripe', verifyStripe, async (req, res) => {
  const event = req.stripeEvent;

  // Return 200 langsung – processing di background
  res.status(200).json({ received: true });

  // Queue processing
  webhookQueue.enqueue(event.id, 'stripe', async () => {
    await processStripeEvent(event);
  });
});

Timeout Protection:

// utils/webhook-timeout.js
/**
 * Handler dengan timeout protection
 * Gateway timeout setelah 10 detik, jadi kita harus selesai < 5 detik
 */
async function processWithTimeout(promise, timeoutMs = 4000) {
  const timeout = new Promise((_, reject) =>
    setTimeout(() => reject(new Error('Webhook processing timeout')), timeoutMs)
  );

```

```

    try {
      return await Promise.race([promise, timeout]);
    } catch (error) {
      // Log untuk debug
      console.error('Webhook processing exceeded timeout:', error.message);
      // Jangan throw – gateway sudah return 200
      return { timeout: true };
    }
  }

  // Penggunaan
  router.post('/stripe', verifyStripe, async (req, res) => {
    const event = req.stripeEvent;
    res.status(200).json({ received: true });

    processWithTimeout(processStripeEvent(event), 4000);
  });

```

5. PCI Compliance Basics

5.1 Apa itu PCI DSS? Payment Card Industry Data Security Standard — standar keamanan untuk perusahaan yang memproses, menyimpan, atau mentransmisikan data kartu kredit.

5.2 Yang BOLEH dan TIDAK BOLEH

☐ Boleh	☐ Tidak Boleh
Menyimpan 6 digit pertama + 4 digit terakhir kartu	Menyimpan full PAN (16 digit)
Menyimpan nama pemegang kartu	Menyimpan CVV/CVC
Menyimpan tanggal kadaluarsa	Menyimpan PIN
Menggunakan token dari payment gateway	Menyimpan track data dari magnetic stripe
Menyimpan alamat tagihan	Mengirim full PAN via email/chat

5.3 Best Practice untuk Aplikasi

```

// services/pci-compliant.js
/**

```

```

* 1. JANGAN PERNAH simpan data sensitif kartu
*
* □ JANGAN:
*   await db('cards').insert({
*     full_pan: '4111111111111111', // □
*     cvv: '123', // □
*     pin: '123456', // □
*   });
*
* □ BOLEH:
*   await db('cards').insert({
*     masked_pan: '411111xxxxxx1111', // □ – hanya 6+4 digit
*     cardholder_name: 'Budi Santoso', // □
*     expiry_month: 12, // □
*     expiry_year: 2027, // □
*     token: 'tok_xxx', // □ – token dari gateway
*   });
*/

/**
* 2. Gunakan token dari payment gateway
*
* Midtrans: token dari snap / core api
* Stripe: token dari stripe.js / Elements
* Xendit: token dari xendit-js
*/
async function chargeWithToken(gatewayToken, amount) {
  // Token sudah aman – tidak perlu simpan data kartu
  const charge = await stripe.paymentIntents.create({
    amount: amount * 100,
    currency: 'idr',
    payment_method: gatewayToken,
    confirm: true,
  });

  return charge;
}

/**
* 3. Logging – jangan log data sensitif
*/
function safeLog(data) {
  const sensitiveFields = ['card_number', 'cvv', 'cvc', 'pin', 'password'];

  const sanitized = { ...data };
  for (const field of sensitiveFields) {

```

```

    if (sanitized[field]) {
      sanitized[field] = '***REDACTED***';
    }
  }

  console.log(JSON.stringify(sanitized));
}

/**
 * 4. Gunakan HTTPS – wajib
 *    (di handle oleh reverse proxy / load balancer)
 */

```

5.4 SAQ (Self-Assessment Questionnaire) Tergantung bagaimana kamu menerima pembayaran:

Metode	SAQ Type	Scope
Stripe Checkout (redirect)	SAQ A	Paling ringan — tidak sentuh data kartu
Stripe Elements (embedded)	SAQ A	Via iframe — tidak sentuh data kartu
Midtrans Snap (redirect/iframe)	SAQ A	Sama — via Snap UI
API langsung (collect card data)	SAQ D	Paling berat — harus compliance penuh

Rekomendasi: Selalu gunakan payment gateway yang handle UI pembayaran (Snap, Checkout Session, Elements). Dengan begitu aplikasi kamu tidak pernah menyentuh data kartu — SAQ A cukup.

□ Latihan

Latihan 1: Database Migration

Buat dan jalankan migration:

1. Copy SQL migration ke file `migrations/001_create_payment_tables.sql`
2. Buat database PostgreSQL (pakai Docker: `docker run --name payment-db -e POSTGRES_PASSWORD=pass -p 5432:5432 -d postgres`)
3. Jalankan migration

4. Insert 1 transaksi test dengan INSERT
5. Query dan verifikasi semua tabel

Output: 5 tabel (transactions, refunds, invoices, webhook_events, payment_attempts) siap di PostgreSQL.

Latihan 2: Idempotency + Race Condition Handler

Implementasi webhook handler dengan race protection:

1. Buat function `handleWebhookEvent(eventId, gateway, processFn)` dengan transaksi atomic
2. Gunakan `INSERT ... ON CONFLICT` untuk mencegah duplicate processing
3. Gunakan optimistic lock (`UPDATE ... WHERE status = 'pending'`) untuk update transaksi
4. Test dengan 2 concurrent request untuk event yang sama
5. Verifikasi hanya 1 yang diproses

Output: Webhook handler aman dari race condition. Event duplikat di-skip.

Latihan 3: Reconciliation Cron

Buat cron job reconciliation:

1. Buat file `cron/reconciliation.js` dengan `node-cron`
2. Ambil transaksi status pending yang sudah > 1 jam
3. Cek status real ke payment gateway
4. Update di database jika berbeda
5. Log statistik (matched/updated/failed)
6. Jalankan manual untuk test: `node cron/reconciliation.js`

Output: Cron job siap, bisa sync status transaksi antara database dan gateway.

Latihan 4: PCI Compliance Checklist

Audit aplikasi payment:

1. Cek kode: apakah ada penyimpanan full PAN atau CVV? (search `card_number`, `cvv`, `cvc` di codebase)
2. Cek logging: apakah ada log yang menyertakan data kartu?
3. Pastikan semua endpoint payment pakai HTTPS
4. Pastikan webhook menggunakan signature verification
5. Buat checklist PCI compliance untuk dokumentasi

Output: Checklist PCI compliance terisi, codebase bebas dari penyimpanan data sensitif.

□ Ringkasan

- **DB Schema:** 5 tabel (transactions, refunds, invoices, webhook_events, payment_attempts) dengan index dan constraint
- **Race Condition:** Gunakan optimistic lock + atomic transaction untuk mencegah data inconsistency
- **Idempotency:** INSERT ON CONFLICT + webhook_events table untuk mencegah duplicate processing
- **Reconciliation:** Cron job periodic untuk sync status antara database dan payment gateway
- **Webhook Timeout:** Return 200 langsung, proses di background queue
- **PCI Compliance:** Jangan simpan CVV/full PAN, gunakan token dari payment gateway, prefer SAQ A dengan redirect/iframe flow